
1. Brainstorming & Ideation

- **Domain Selection**: Fintech and automated credit risk evaluation.
- **Problem Statement**: Banks and financial institutions face high default rates due to manual and sub-optimal credit risk evaluation processes.
- **Proposed Solution**: An end-to-end machine learning solution that aggregates applicant demographics and repayment history records to predict the probability of client default.
- **Key Deliverable**: A responsive, self-hosted web application and dynamic dashboard enabling loan officers to evaluate applicant data instantly.

2. Requirement Analysis

- **Functional Requirements**:
 - Automatically fetch raw application and credit card transaction data.
 - Preprocess and align inputs (handling class imbalances and missing values).
 - Train multiple classifier models and select the highest performing champion model.
 - Host a lightweight Flask web API with an interactive web UI.
- **Non-Functional Requirements**:
 - Real-time scoring latency (< 1s per request).
 - Serialization of ML pipelines to avoid data leakage.
 - Automated deployment support via Render or other cloud provider blueprint formats.
- **Tools & Libraries**: Python, Pandas, Scikit-learn, XGBoost, Flask, Joblib, Gunicorn, HTML5, CSS3, JavaScript.

3. Project Design Phase

- **System Architecture**:

- **Data Layer**: Programmatic retrieval and target-label creation using transactional records (deriving status 1 for 30+ days overdue).
- **Pipeline Layer**: Modular preprocessing using Scikit-Learn `ColumnTransformer` (scaling numerical fields and one-hot encoding categories).
- **Model Layer**: Logistic Regression, Decision Tree, XGBoost, and Random Forest ensemble models.
- **Presentation Layer**: Flask-powered classic corporate dashboard featuring interactive forms and real-time visualization of model metrics.
- **UI Mockup / Flow**: Dashboard contains an input form on the left pane and metrics/plots on the right pane. Results page uses an animated circular credit meter (green/orange/red gauge) based on default probability.

4. Project Planning Phase

- **Milestones**:
 - [x] Phase 1: Environment Setup & Raw Data Ingestion.
 - [x] Phase 2: Feature Engineering & Preprocessing Pipeline Design.
 - [x] Phase 3: Model Grid Search, Optimization, and Champion Serialization.
 - [x] Phase 4: Flask Backend & Corporate Slate CSS Web Interface.
 - [x] Phase 5: Build Script Automation & Cloud Configuration.
 - [x] Phase 6: Model Validation & Deployment Testing.

5. Project Development Phase

- **Core Modules**:
 - [src/data_pipeline.py](file:///c:/Users/J.ABHILASH/credit-card-approval-prediction/src/data_pipeline.py): Performs repayment target engineering and merges CSV datasets.
 - [src/data_preprocessing.py](file:///c:/Users/J.ABHILASH/credit-card-approval-prediction/src/data_preprocessing.py): Implements stratified splitting and exports `preprocessor.joblib`.
 - [src/train_models.py](file:///c:/Users/J.ABHILASH/credit-card-approval-prediction/src/train_models.py): Performs grid-search cross-validation over classifiers.
 - [app.py](file:///c:/Users/J.ABHILASH/credit-card-approval-prediction/app.py): Backend web API providing model inference endpoints.

- [static/css/style.css](file:///c:/Users/J.ABHILASH/credit-card-approval-prediction/static/css/style.css): Slate styling rules for dashboard grids and panels.

6. Project Testing

- **Model Assessment**: Models were evaluated on the test partition. Performance comparison:

Model Classifier	Accuracy	Precision	Recall	F1-Score	ROC-AUC
	---	---	---	---	---
Logistic Regression	57.89%	13.66%	48.48%	0.2132	0.5501
Decision Tree	67.40%	21.22%	65.27%	0.3203	0.7181
XGBoost	78.77%	29.56%	58.16%	0.3920	0.7510
Random Forest (Champion)	80.44%	30.99%	53.96%	0.3937	0.7559

- **Functional Testing**: Validated Flask endpoints with client-side form assertions. Added boundaries to handle missing demographic fields gracefully.

7. Project Documentation

- **Local Installation**:

1. Clone this repository and navigate to its folder.
2. Run `python -m venv venv` and activate the virtual environment.
3. Execute `pip install -r requirements.txt` to install package dependencies.
4. Run `python build\_assets.py` to compile preprocessing components and train the model weights.
5. Run `python app.py` to launch the local web server at `http://127.0.0.1:5000`.

- **Cloud Deployment**:

- The project is configured for one-click deployment using Render Blueprints (`render.yaml`). Connect your GitHub repo to Render and launch a Blueprint instance.

8. Project Demonstration

- **Interactive UI Walkthrough**:

- Access the home dashboard. The left pane provides input forms for demographics (Gender, Income, Education, Housing, Job Details).
- The right pane displays EDA metrics and pre-computed classifier comparison charts.
- Submitting an applicant profile redirects to the results panel, showcasing a visual credit score gauge meter (expressing approval risk as Good or Delinquent risk score percentages).
- ****Video / Slides Reference****: Team deliverables, demonstration video links, and slidedeck files can be linked here to replace placeholder entries.

Note: Replace the placeholder files with your team's project deliverables.